



GitHub Actions Certification

Complete Study Notes & Exam Guide

Notes compiled from the freeCodeCamp / ExamPro
“GitHub Actions Certification – Full Course to PASS the Exam” video,
combined with general GitHub Actions reference knowledge.

Contents

- Exam overview & domains · Workflow syntax & components
- Events & triggers · Runners · Secrets & variables
- Contexts, expressions & conditionals · Caching & artifacts
- Custom actions (JS / Docker / Composite) · Reusable workflows
- Enterprise administration · Exam-ready cheat sheets

Table of Contents

1. Course & Exam Overview	3
2. Core Concepts: Workflows, Jobs, Steps & Actions	5
3. Anatomy of a Workflow File (YAML)	7
4. Events & Triggers	9
5. Manual, Repository Dispatch & External Webhook Events	11
6. Access Tokens & Account Permissions	13
7. Conditional Keywords & Expressions	14
8. Contexts & Workflow Commands	16
9. Runners: GitHub-Hosted vs. Self-Hosted	18
10. Secrets, Variables & GITHUB_TOKEN	20
11. Packages, Container Registry & Releases	22
12. Service Containers, Matrix Builds & Runner Routing	23
13. Caching & Artifacts	24
14. Environments, Protection Rules & Workflow Management	25
15. Authoring Custom Actions	26
16. Reusable Workflows & Enterprise Administration	28
17. Exam Cheat Sheet & Final Tips	29

NOTE: This document is built from two sources: (1) on-screen slide and code content sampled throughout the uploaded video's 3 hour 10 minute runtime, which follows the ExamPro/freeCodeCamp GitHub Actions certification curriculum and its **Github-Examples** demo repository, and (2) general reference knowledge of GitHub Actions to fill in explanation, syntax detail and context around what the video demonstrates. Where the video's exact wording could not be fully captured, the concept and official behavior are explained accurately rather than guessed at.

1. Course & Exam Overview

The video is the **freeCodeCamp / ExamPro “GitHub Actions Certification – Full Course to PASS the Exam.”** It opens with an exam-prep briefing before moving into a fully hands-on walkthrough of GitHub Actions inside a real repository (**ExamProCo/Github-Examples**), and closes by building a custom GitHub Action from scratch and publishing it.

1.1 What the GitHub Actions Certification Covers

GitHub Actions is described as the **programming-focused GitHub certification**. Unlike GitHub Foundations (which is conceptual/non-technical), this exam expects hands-on comfort with YAML and CI/CD thinking. The unofficial course code used throughout is **“ACTIONS.”** The course stresses that GitHub Actions is popular as a CI/CD tool because of its free tier for public repos, ease of use, flexibility, and its native, no-extra-setup integration with GitHub repositories.

- Creating CI/CD or build pipelines
- Writing YAML workflow definitions
- Creating custom GitHub Actions
- Configuring workflow environments
- Securely passing data to workflows (secrets & variables)

1.2 The GitHub Certification Roadmap

GitHub's certifications are organized into tiers. GitHub Actions sits at the **Level 100 (Associate)** tier alongside GitHub Foundations, below Level 150/200 role-based certifications (e.g. Cloud Developer, Cloud Security Engineer, Enterprise Engineer) and the Level 300 Advanced tier. The course notes an important caveat repeated throughout: **GitHub certifications validate platform knowledge, not general programming, architecture, or broader technical ability** — they certify that you know the product, not that you can code or design systems.

1.3 Who Should Take It / How Long to Study

The course frames readiness on a spectrum:

Beginner profile	Experienced profile
Never used GitHub Actions	Practical, hands-on GitHub Actions + YAML experience
Never written YAML or code before	Working technical knowledge of CI/CD concepts
No DevOps exposure	Strong existing DevOps background

The recommended prep path is three-fold: **(1)** watch the lecture and memorize key facts, **(2)** do hands-on labs inside your own GitHub account following along, and **(3)** take paid practice exams that simulate the real test (ExamPro's own being the one promoted in the course).

1.4 Exam Format

Attribute	Detail
Total questions	75

Attribute	Detail
Scored questions	60
Unscored questions	15 (do not affect your score)
Delivery	PSI-proctored: online (remote) or in-person at a PSI test center
Question types	Multiple choice / multiple answer

NOTE: Unscored questions exist to trial new questions, calibrate difficulty/passing score, and catch people who memorized leaked “dump” questions. If you hit a question that feels unfamiliar or unreasonably hard, it may well be one of the 15 unscored ones — stay calm and move on rather than assuming you are failing.

1.5 Exam Domains & Weighting

The exam content outline is split into four weighted domains. Domain weighting tells you roughly how many of the 75 questions will come from each area, and is the single most useful map for prioritizing study time:

Domain	Weight
Domain 1: Author and maintain workflows	40%
Domain 2: Consume workflows	20%
Domain 3: Author and maintain actions	25%
Domain 4: Manage GitHub Actions for the enterprise	15%

Domain 1 — Author and maintain workflows (40%)

- Identify correct workflow-job YAML syntax (indentation, nesting/encapsulation of jobs/steps).
- Use job steps for both actions and raw shell commands.
- Use conditional keywords on steps (`if`).
- Describe how actions, workflows, jobs, steps, runs, and the Marketplace relate to one another.
- Identify scenarios suited to GitHub-hosted vs. self-hosted runners.
- Implement workflow commands as a run step to communicate with the runner.
- Demonstrate use of dependent jobs (needs).
- Use encrypted secrets and environment variables in a workflow, including identifying default environment variables available during workflow construction.

Domain 2 — Consume workflows (20%)

Covers interpreting workflow runs, reading logs/annotations, understanding artifacts and re-running/cancelling jobs from the consumer side (explored hands-on throughout the demo sections of the video).

Domain 3 — Author and maintain actions (25%)

- Implement workflow commands *within an action* to talk to the runner (including exit codes).
- Select an appropriate distribution model for an action (public, private, or Marketplace).
- Apply best practices for distributing custom actions.

- Create a release/versioning strategy for an action.
- Publish an action to the GitHub Marketplace.

Domain 4 — Manage GitHub Actions for the enterprise (15%)

Covers enterprise/organization-level governance: allow-listing actions, managing self-hosted runners centrally, artifact/log retention policy, and controlling who can approve workflow runs from forks.

2. Core Concepts: Workflows, Jobs, Steps & Actions

The video's hands-on section begins here, using the **ExamProCo/Github-Examples** repository. GitHub Actions is introduced as a **CI/CD pipeline built directly into your GitHub repository**, used to automate things like running test suites, building container images, compiling static sites, and deploying code to servers.

2.1 The Object Hierarchy

Everything in GitHub Actions nests inside a strict hierarchy, and knowing this hierarchy cold is one of the highest-value things for Domain 1:

Level	What it is
Workflow	A YAML file in <code>.github/workflows/</code> describing an automated process, triggered by events
Job	A set of steps that execute on the same runner; jobs run in parallel by default
Step	A single task: either runs shell commands or invokes an action
Action	A reusable, packaged unit of code (JS, Docker, or Composite) referenced from a step
Runner	The server (VM or container) that actually executes a job's steps
Marketplace	GitHub's public catalog of shareable, reusable actions and workflows

A **workflow** contains one or more **jobs**; each job runs on one **runner** and contains an ordered list of **steps**; each step either runs a shell command (`run:`) or calls a reusable **action** (`uses:`). Actions themselves can be published to and pulled from the **Marketplace**.

2.2 Where Workflows Live

GitHub Actions workflow files are YAML files that must live in the `.github/workflows/` directory at the root of a repository. Each `.yaml` file in that folder is an independent workflow. Any repository also exposes a dedicated **Actions** tab in the GitHub UI, which lists all workflows, their run history, statuses, logs, and re-run/cancel controls — this is the primary place the video uses to inspect workflow runs throughout the demos.

GitHub also ships starter **workflow templates** (e.g. “Deploy Node.js to Azure Web App”) you can pick from when creating a new workflow, which scaffold a working `.yaml` file for common languages and deployment targets.

2.3 A Minimal Workflow

```

name: Example Workflow

on:
  push:
    branches: [ "$default-branch" ]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v3

      - name: Run a script
        run: echo "This is a step"

```

Reading this top to bottom: `name` labels the workflow in the Actions UI; `on` declares the event trigger(s); `jobs` defines one or more named jobs (here, `build`); `runs-on` picks the runner image; and each list item under `steps` is executed top-to-bottom in order.

2.4 Actions Permissions (Repository Settings)

Under **Settings** → **Actions** → **General**, a repository (or organization) controls which actions are even allowed to run:

- **Allow all actions and reusable workflows** — any action from anywhere, including the public Marketplace.
- **Allow <org> actions and reusable workflows** — restrict to actions defined within the organization.
- **Allow <org> and select non-<org> actions and workflows** — org actions plus an explicit allow-list of external actions/patterns.
- **Disable actions** — turn Actions off entirely for the repository.

The same settings page controls **artifact and log retention** (a configurable number of days, default commonly 90) and **fork pull request workflow approval policy** — i.e., whether workflow runs triggered by pull requests from outside collaborators/forks require a maintainer to manually approve the run before it executes. Options include requiring approval for first-time contributors, for all outside collaborators, or for all external contributors — this exists specifically to stop malicious forks from exfiltrating secrets or burning compute via automatically-triggered workflow runs.

NOTE: Fork-PR approval gating is a favorite exam trap: remember that workflows triggered from a fork's pull request do **not** get access to repository secrets unless the run has been approved (and even then, certain protections still apply).

3. Anatomy of a Workflow File (YAML)

This section expands on **Workflow Components**, one of the video's own slide headings, going key-by-key through what can appear in a workflow file.

3.1 Top-Level Keys

Key	Purpose
name	Display name of the workflow shown in the Actions tab
on	Event(s) that trigger the workflow
env	Workflow-level environment variables, inherited by all jobs
permissions	Sets the default GITHUB_TOKEN permission scopes for the workflow
jobs	The map of job-id → job definition that does the actual work
concurrency	Limits/queues simultaneous runs sharing a concurrency group
defaults	Default settings (e.g. shell, working-directory) applied to all run steps

3.2 Job-Level Keys

Key	Purpose
runs-on	Which runner/image the job executes on (e.g. ubuntu-latest, self-hosted)
needs	Declares this job depends on one or more other jobs completing first
if	Conditional expression controlling whether the job runs at all
strategy.matrix	Runs the job multiple times across a matrix of variable combinations
environment	Ties the job to a named deployment environment (and its protection rules)
outputs	Values the job exposes for downstream/dependent jobs to consume
steps	Ordered list of steps the job executes

3.3 Step-Level Keys

Key	Purpose
name	Human-readable label for the step in logs/UI
uses	Invoke a reusable action, e.g. actions/checkout@v4
run	Execute one or more shell commands directly
with	Input parameters passed to an action used via 'uses'
env	Step-scoped environment variables

Key	Purpose
if	Conditional expression controlling whether this single step runs
id	Identifier so later steps can reference this step's outputs
continue-on-error	Let the job keep running even if this step fails

3.4 Indentation & Encapsulation

Because workflow files are YAML, structure is defined entirely by indentation (spaces, never tabs). The exam explicitly tests the ability to read whether a key sits under jobs, under a specific job, or under steps purely from its indent level. steps is a YAML sequence (each entry starts with -), while jobs and each job definition are YAML mappings (key: value).

```
jobs:
  job_one:                # mapping key = job id
    runs-on: ubuntu-latest
    steps:                 # sequence starts here
      - name: Step A       # first list item (a mapping itself)
        run: echo "A"
      - name: Step B
        run: echo "B"
  job_two:                 # sibling job, same indent as job_one
    runs-on: ubuntu-latest
    needs: job_one
    steps:
      - run: echo "runs after job_one"
```

3.5 Dependent (Sequential) Jobs

By default, all jobs in a workflow run **in parallel**. To force ordering, a job declares needs, naming one or more job ids it must wait for. This also lets a downstream job read the upstream job's declared outputs.

```
jobs:
  build:
    runs-on: ubuntu-latest
    outputs:
      build-id: ${ steps.set-id.outputs.id }
    steps:
      - id: set-id
        run: echo "id=123" >> "$GITHUB_OUTPUT"

  deploy:
    needs: build
    runs-on: ubuntu-latest
    steps:
      - run: echo "Deploying build ${ needs.build.outputs.build-id }"
```

4. Events & Triggers

The video's “Different Types of GitHub Actions” and “Triggering Schedule Event” / “Triggering Single or Multiple Events” sections cover the on: key in depth: an **event trigger** is what causes a workflow to run at all. GitHub Actions supports 35+ distinct event triggers.

4.1 Common Event Types

- **push** — runs on any push to the repo (optionally filtered by branches/paths/tags).
- **pull_request** — runs when PRs are opened, updated (synchronized), or otherwise change state.
- **issues** — runs on issue activity such as creation or labeling.
- **release** — runs when a new release is published.
- **schedule** — runs on a cron schedule.
- **workflow_dispatch** — manual trigger via the GitHub UI/API.
- **repository_dispatch** — externally triggered via the REST API/webhook from outside GitHub.

4.2 Push Events

```
on:
  push:
    branches:
      - main
      - "releases/**"
    paths:
      - "src/**"
```

Filters like branches, branches-ignore, paths, and paths-ignore scope exactly when a push-triggered workflow should fire, which keeps CI from running on every unrelated change.

4.3 Scheduled (Cron) Events

The **schedule** trigger uses standard 5-field **POSIX cron syntax** (minute, hour, day-of-month, month, day-of-week), evaluated in UTC. The video demonstrates this directly against GitHub's own `crontab.guru`-style mental model.

```
on:
  schedule:
    - cron: "0 3 * * *" # every day at 03:00 UTC
```

NOTE: Scheduled workflows only run on the **default branch**, and GitHub may delay or drop a scheduled run during periods of high load — the schedule is a best-effort trigger, not a guaranteed-to-the-minute one. Also, scheduled workflows are automatically disabled after 60 days of repository inactivity.

4.4 Triggering on Multiple Events

A single workflow can be triggered by several different events at once by listing them under on, each optionally with its own filters:

```
on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]
  workflow_dispatch:
```

Inside the workflow, the built-in `github.event_name` context value tells you which of the listed events actually fired for a given run — useful for branching logic when one workflow handles several trigger types.

5. Manual, Repository Dispatch & External Webhook Events

5.1 Manual Events — workflow_dispatch

The workflow_dispatch trigger adds a “**Run workflow**” button in the Actions tab, letting a human kick off the workflow on demand from a chosen branch. It optionally accepts typed inputs (string, boolean, choice, environment) which become available via the github.event.inputs context, or more directly through the inputs context.

```
on:
  workflow_dispatch:
    inputs:
      environment:
        description: "Target environment"
        required: true
        type: choice
        options: [ staging, production ]
      debug:
        description: "Enable debug logging"
        type: boolean
        default: false

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - run: echo "Deploying to ${ inputs.environment }"
```

5.2 Repository Dispatch

repository_dispatch lets an **external system** trigger a workflow by calling the GitHub REST API directly, rather than through GitHub's own UI. It requires a Personal Access Token (or GitHub App token) with appropriate repo scope, and the caller supplies a custom event_type string plus an optional JSON payload the workflow can inspect via github.event.client_payload.

```
# From an external system:
curl -X POST \
  -H "Accept: application/vnd.github+json" \
  -H "Authorization: Bearer $TOKEN" \
  https://api.github.com/repos/OWNER/REPO/dispatches \
  -d '{"event_type":"deploy-requested","client_payload":{"env":"prod"}}'
```

```
on:
  repository_dispatch:
    types: [ deploy-requested ]

jobs:
  handle:
    runs-on: ubuntu-latest
    steps:
      - run: echo "Env = ${ { github.event.client_payload.env } }"
```

5.3 External Webhook Events

The course demonstrates using GitHub webhooks together with an external listener (a small Codespaces-hosted service in the video, shown against real repository events such as commits) to show how outside systems can react to GitHub activity, and conversely how GitHub Actions itself can be driven by outside systems via `repository_dispatch`. The key distinction for the exam:

- **GitHub webhooks** — push data *out* of GitHub to an external URL when repo events occur.
- **repository_dispatch** — pushes a trigger *into* GitHub Actions from an external caller.

5.4 Account Permissions & Tokens Used for Dispatch

Triggering workflows from outside GitHub, or letting GitHub Actions call back out to other GitHub APIs, requires an authentication token. The video walks through creating both classic and **fine-grained personal access tokens (PATs)** under account developer settings, and Codespaces-based examples of using such a token to call the dispatch API. Fine-grained PATs let you scope a token to specific repositories and specific permission categories (contents, issues, actions, etc.) rather than the broad all-repo scopes of classic PATs — the security-conscious, GitHub-recommended default going forward.

6. Access Tokens & Account Permissions

This section consolidates the video's **Account permissions** and personal-access-token walkthroughs.

Token type	Notes
GITHUB_TOKEN	Auto-generated per workflow run; scoped to that repo; expires when the job ends
Classic PAT	User-wide scopes (repo, workflow, etc.); broad access across all your repos
Fine-grained PAT	Scoped to chosen repositories and specific permission categories, with an expiry date
GitHub App token	Installed per-repo/org; short-lived; best for automation acting 'as an app'

NOTE: For the exam: know that the default GITHUB_TOKEN is automatically created and destroyed per workflow run and needs no manual setup, whereas PATs and webhook secrets must be generated by a user and stored (typically as an encrypted secret) before a workflow or external system can use them.

7. Conditional Keywords & Expressions

7.1 The if Keyword

Both jobs and steps accept an `if` key. If the expression evaluates to `false`, that job/step is skipped (shown as skipped in the run, not failed). Expressions are wrapped in `${{ }}`, though on a bare `if:` line GitHub allows omitting the double-braces.

```
steps:
  - name: Only on main
    if: github.ref == 'refs/heads/main'
    run: echo "on main branch"

  - name: Only if previous step failed
    if: failure()
    run: echo "cleanup after failure"

  - name: Always run, even if job failed
    if: always()
    run: echo "always runs"
```

7.2 Built-in Status Check Functions

Function	True when...
<code>success()</code>	All previous steps succeeded (this is the implicit default if no 'if' is set)
<code>failure()</code>	Any previous step in the job has failed
<code>cancelled()</code>	The workflow run was cancelled
<code>always()</code>	Always evaluates true — runs regardless of prior outcome

NOTE: A step or job with no explicit 'if' behaves as though `if: success()` were set — that's why a single earlier failure normally stops the rest of the job from running unless you explicitly use `always()` or `failure()`.

7.3 Expressions & Operators

GitHub Actions expressions support comparison operators (`==`, `!=`, `<`, `>`), logical operators (`&&`, `||`, `!`), and built-in functions such as `contains()`, `startsWith()`, `endsWith()`, `join()`, and `format()`.

```
if: contains(github.event.head_commit.message, '[skip ci]') == false
```

8. Contexts & Workflow Commands

8.1 Workflow Contexts

A **context** is a named object exposing information about the run, accessed with `${{ context.property }}` syntax. The video's “**Workflow Context**” section walks these live against actual runs:

Context	Provides
github	Event payload, repo, ref, sha, actor, run id/number, etc.
env	Environment variables set at workflow/job/step level
job	Status/container info for the currently running job
steps	Outputs and outcomes of previous steps in the same job (by step id)
runner	Info about the runner executing the job (OS, temp dir, arch)
secrets	Configured encrypted secrets available to the workflow
vars	Configuration variables (non-secret) available to the workflow
needs	Outputs/results of jobs this job depends on
matrix	The current combination of values in a strategy matrix job
inputs	Inputs supplied to workflow_dispatch or a reusable/called workflow

8.2 Workflow Commands

Workflow commands are special strings a step can print to stdout to communicate back to the runner — setting outputs, environment variables, adding log annotations, masking secrets, or grouping log lines. The video demonstrates several of these directly inside `run: steps`.

```

steps:
  - name: Set an output for later steps
    id: info
    run: echo "greeting=Hello" >> "$GITHUB_OUTPUT"

  - name: Set an env var for later steps
    run: echo "MY_VAR=example" >> "$GITHUB_ENV"

  - name: Add to PATH for later steps
    run: echo "/custom/bin" >> "$GITHUB_PATH"

  - name: Group related log lines
    run: |
      echo "::group::Install dependencies"
      npm install
      echo "::endgroup::"

  - name: Emit an error annotation
    run: echo "::error file=app.js,line=10::Something went wrong"

  - name: Mask a value in logs
    run: echo "::add-mask::${{ secrets.MY_SECRET }}"

  - name: Use the earlier output
    run: echo "${{ steps.info.outputs.greeting }}, world"

```

NOTE: Older syntax such as `::set-output name=x::value` and `::set-env name=x::value` is **deprecated**. The current, exam-correct approach is writing to the special environment files `$GITHUB_OUTPUT`, `$GITHUB_ENV`, and `$GITHUB_PATH` as shown above.

8.3 Exit Codes

A step (whether a shell command or a custom action) signals success or failure to the runner purely through its **process exit code**: 0 means success, and any non-zero value fails the step (and by default, the job). Custom JavaScript actions can call `core.setFailed(message)` to fail deliberately with a clear message; Docker actions fail simply by having their entrypoint process exit non-zero.

9. Runners: GitHub-Hosted vs. Self-Hosted

9.1 GitHub-Hosted Runners

GitHub-hosted runners are ephemeral virtual machines GitHub provisions on demand, used with keywords like `ubuntu-latest`, `windows-latest`, and `macos-latest`. Each job gets a clean VM, pre-installed with a large set of common tooling, that is destroyed after the job finishes.

- Zero maintenance — GitHub manages the OS, patches, and tooling images.
- Clean/isolated environment on every run (no leftover state between runs).
- Usage-based billing: free minutes on public repos and a monthly quota on private repos, billed per-minute beyond that, with a multiplier depending on OS (Windows/macOS cost more per minute than Linux).

9.2 Self-Hosted Runners

A self-hosted runner is a machine (physical, VM, container, on-prem or cloud) that you register yourself against a repository, organization, or enterprise, running the GitHub Actions runner agent. The video demonstrates registering a fresh self-hosted runner end-to-end via **Settings** → **Actions** → **Runners** → **New self-hosted runner**, which generates a one-time download-and-configure script.

```
# example pattern shown for registering a self-hosted runner
./config.sh --url https://github.com/OWNER/REPO --token REGISTRATION_TOKEN
./run.sh
```

- You control the hardware, OS, pre-installed software, and persistence between runs.
- Useful for accessing internal/private network resources GitHub-hosted runners can't reach.
- Useful for specialized hardware (GPUs) or licensing that GitHub-hosted images don't provide.
- You are responsible for security patching, scaling, and cleanup between jobs.

EXAM TIP: Because self-hosted runners are not ephemeral by default, running them against **public repositories** is explicitly discouraged by GitHub — anyone who can open a pull request could potentially run arbitrary code on your infrastructure. This is a recurring exam point.

9.3 Routing a Workflow to a Specific Runner

`runs-on` can target self-hosted runners either by the literal `self-hosted` label or by custom labels assigned when the runner was registered (e.g. `environment`, `hardware type`). Using an array requires the runner to match **all** listed labels.

```
jobs:
  build:
    runs-on: [ self-hosted, linux, x64, gpu ]
```

9.4 Configuring Self-Hosted Runners for Enterprise

At the organization or enterprise level, self-hosted runners can be pooled into **runner groups**, which control which repositories/organizations are allowed to use a given set of runners — letting platform teams centrally provision runner capacity while restricting which teams may consume it. This is core Domain 4 material.

10. Secrets, Variables & GITHUB_TOKEN

10.1 Encrypted Secrets

Secrets are encrypted values (API keys, credentials, tokens) stored at the repository, environment, or organization level under **Settings** → **Secrets and variables** → **Actions**. They are encrypted at rest, only decrypted for a workflow run that explicitly references them, and GitHub automatically **redacts (masks)** any matching secret value that happens to be printed to logs.

```
steps:
  - name: Use a secret
    env:
      API_KEY: ${ secrets.API_KEY }
    run: curl -H "Authorization: Bearer $API_KEY" https://example.com
```

NOTE: Secrets are never passed automatically — you must reference `secrets.NAME` explicitly. They also cannot be used directly inside an `'if:'` condition on the job/step level for security reasons in some contexts, and are not available at all to workflows triggered from forked-repo pull requests unless approved.

10.2 Configuration Variables

Alongside secrets, GitHub Actions supports **configuration variables** (vars context) — non-sensitive, plain-text values stored the same way as secrets but visible in the UI and in logs. They're the right tool for things like a default region name or a feature flag that doesn't need to be encrypted.

```
steps:
  - run: echo "Deploying to ${ vars.DEPLOY_REGION }"
```

10.3 Custom Environment Variables

Environment variables can be set at three scopes — workflow-level (applies to every job), job-level (applies to every step in that job), and step-level (applies to just that step) — with the more specific scope overriding a broader one of the same name.

```
env:
  GLOBAL_VAR: "workflow-scope"

jobs:
  build:
    runs-on: ubuntu-latest
    env:
      JOB_VAR: "job-scope"
    steps:
      - env:
          STEP_VAR: "step-scope"
        run: echo "$GLOBAL_VAR $JOB_VAR $STEP_VAR"
```

Env vars can also be set dynamically at runtime from inside a step by writing to `$GITHUB_ENV`, as covered in Section 8.2, which makes them available to *subsequent* steps in the same job (not the step that set them).

10.4 The GITHUB_TOKEN Secret

GITHUB_TOKEN is a special, automatically generated secret injected into every workflow run — no setup needed. It authenticates as `github-actions[bot]` and is scoped to just the repository the workflow runs in. Its default permission scopes can be tightened (or broadened, up to a repository's allowed maximum) with a `permissions:` block at the workflow or job level.

```
permissions:
  contents: read
  issues: write
  pull-requests: write
```

NOTE: GITHUB_TOKEN expires automatically when the job finishes (or after a hard ceiling of 24 hours), so it can never be used for long-lived automation — that's what PATs or GitHub App tokens are for.

11. Packages, Container Registry & Releases

This section follows the video's “Publish GitHub Package,” “Publish GitHub Container Registry,” and “Publish Component as GitHub Release / Deploy Release to Cloud Provider” demos.

11.1 GitHub Packages

GitHub Packages hosts package registries (npm, Maven, NuGet, RubyGems, and more) directly alongside your repository. A workflow can build and publish a package on every release using the standard package-manager CLI, authenticated with GITHUB_TOKEN.

```
- name: Publish npm package
  run: npm publish
  env:
    NODE_AUTH_TOKEN: ${ secrets.GITHUB_TOKEN }
```

11.2 GitHub Container Registry (GHCR)

ghcr.io is GitHub's Docker/OCI container image registry. A typical workflow logs in, builds an image, tags it against the GHCR namespace, and pushes it.

```
- name: Log in to GHCR
  uses: docker/login-action@v3
  with:
    registry: ghcr.io
    username: ${ github.actor }
    password: ${ secrets.GITHUB_TOKEN }

- name: Build and push image
  run: |
    docker build -t ghcr.io/${ github.repository }:latest .
    docker push ghcr.io/${ github.repository }:latest
```

11.3 Publishing a GitHub Release

A release packages a tagged version of your code (plus optional binary assets) as a distributable artifact on the repo's Releases page. Workflows commonly trigger on: `release: types: [published]` to react to a new release, or use an action like `softprops/action-gh-release` to create the release itself as part of a workflow.

11.4 Deploying a Release to a Cloud Provider

The video closes this arc by wiring a workflow to deploy a built release to a cloud target (e.g. a web app / hosting service), illustrating the common end-to-end CD pattern: `build → test → package/publish → deploy`, usually gated behind a `needs: chain` and/or an `environment: with` its own protection rules (see Section 14).

12. Service Containers, Matrix Builds & Runner Routing

12.1 Service Containers

services spins up additional Docker containers (databases, caches, message brokers) alongside a job's main container/runner, useful for integration tests that need a real Postgres/Redis/etc. instance. Service containers are only available on Linux runners (or when the job itself runs in a container).

```
jobs:
  test:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:16
        env:
          POSTGRES_PASSWORD: postgres
        ports:
          - 5432:5432
        options: >-
          --health-cmd pg_isready
          --health-interval 10s
          --health-timeout 5s
          --health-retries 5
    steps:
      - run: psql -h localhost -U postgres -c 'SELECT 1'
```

12.2 Matrix Strategy

strategy.matrix runs the same job repeatedly against every combination of listed variables (e.g. multiple OS/language versions), letting one job definition fan out into many parallel runs.

```
jobs:
  test:
    strategy:
      matrix:
        os: [ ubuntu-latest, windows-latest ]
        node: [ 18, 20 ]
    runs-on: ${ matrix.os }
    steps:
      - uses: actions/setup-node@v4
        with:
          node-version: ${ matrix.node }
```

This produces 4 parallel job runs (2 OS × 2 Node versions). fail-fast: false can be added under strategy so that one failing matrix combination doesn't cancel the others.

12.3 Routing Workflows to the Right Runner

Beyond the label-matching covered in Section 9.3, routing decisions commonly combine OS choice, custom labels, and (for self-hosted) runner groups scoped to specific repos/teams — letting an organization steer, e.g., GPU-heavy ML jobs to a dedicated pool while normal CI stays on cheap GitHub-hosted Linux runners.

13. Caching & Artifacts

13.1 Caching Dependencies

Because GitHub-hosted runners start from a clean VM every time, dependency installation (npm/pip/Maven etc.) is repeated on every run unless cached. `actions/cache` persists a directory between runs, keyed by a hash (commonly of the lockfile) so the cache invalidates automatically when dependencies change.

```
- name: Cache node modules
  uses: actions/cache@v4
  with:
    path: ~/.npm
    key: ${ runner.os }-npm-${ hashFiles('**/package-lock.json') }
    restore-keys: |
      ${ runner.os }-npm-
```

Many official setup actions (e.g. `actions/setup-node`) also expose a built-in `cache:` input that wraps this pattern automatically.

13.2 Caching Job Dependencies & Build Output

The same `actions/cache` mechanism is used not just for package-manager caches but also for compiled build output between jobs/runs (e.g. a compiled binary, a Docker layer cache) to speed up subsequent CI runs, as long as a suitable cache key strategy is chosen so stale output isn't served.

13.3 Artifacts

Artifacts are files a job explicitly uploads so they can be downloaded from the run summary UI, or passed to a later job in the same workflow run. Unlike caches, artifacts are meant for run outputs (test reports, build binaries, logs) rather than for speeding up dependency installation, and they are retained for a configurable period (governed by the repository/organization retention setting covered in Section 2.4).

```
- name: Upload build output
  uses: actions/upload-artifact@v4
  with:
    name: dist
    path: dist/

# in a later job:
- name: Download build output
  uses: actions/download-artifact@v4
  with:
    name: dist
```

13.4 Removing a Workflow Artifact

Artifacts can be deleted manually from a workflow run's summary page in the UI, via the REST API, or they simply expire automatically once the retention window elapses — all three approaches were shown as options in the video for reclaiming storage.

14. Environments, Protection Rules & Workflow Management

14.1 Deployment Environments

An environment: (e.g. staging, production) can be attached to a job. Environments can hold their own scoped secrets/variables and, crucially, their own **protection rules**.

```
jobs:
  deploy:
    runs-on: ubuntu-latest
    environment:
      name: production
      url: https://example.com
    steps:
      - run: ./deploy.sh
```

14.2 Environment Protection Rules

- **Required reviewers** — one or more specific people/teams must manually approve before the job proceeds.
- **Wait timer** — a mandatory delay (in minutes) before the job is allowed to run, giving time to cancel a bad deploy.
- **Deployment branch/tag rules** — restrict which branches or tags are even allowed to deploy to this environment.

NOTE: When a job targets a protected environment, the run pauses in a 'Waiting' state at that job until reviewers approve it — this is a very common exam scenario for questions about controlling production deploys.

14.3 Disabling vs. Deleting a Workflow

Action	Effect
Disable workflow	Stops future triggers from starting new runs; past run history is preserved and the workflow file remains; can be re-enabled.
Delete workflow file	Removing the .yml from .github/workflows/ (via a commit) stops it from running at all; historical run logs may remain.

NOTE: Disabling is reversible and non-destructive — it is the safer, preferred way to temporarily pause a noisy or broken workflow, versus outright deleting its file from the repository.

15. Authoring Custom Actions

The video's final major arc builds a real custom action from scratch inside a demo project (referred to on-screen as the “**barsoom**” / “**scaling system**” Codespace), covering exactly what Domain 3 of the exam tests.

15.1 The Three Action Types

Type	How it works	Best for
JavaScript action	Runs directly on the runner via Node.js, using the <code>@actions/core</code> and <code>@actions/github</code> toolkits.	Fastest, and @actions/toolkit (no container overhead)
Docker container action	Packages the action as a Docker image; runs the image in a container. Supports the job dependencies; Linux runners only.	Full control over the environment
Composite action	Bundles a sequence of existing 'run' and 'uses' steps.	Wrapping/reusing a repeated sequence of steps without writing a new action.

15.2 Files & Directory Structure

Every action, regardless of type, needs an `action.yml` (or `action.yaml`) metadata file at its root, declaring its name, description, inputs, outputs, and how it runs.

JavaScript action layout

```
my-action/  
  action.yml  
  index.js  
  package.json  
  node_modules/      # or bundled via ncc into a single dist/index.js
```

```
# action.yml  
name: "My Greeting Action"  
description: "Prints a greeting"  
inputs:  
  who-to-greet:  
    description: "Who to greet"  
    required: true  
    default: "World"  
outputs:  
  time:  
    description: "The greeting time"  
runs:  
  using: "node20"  
  main: "index.js"
```

Docker container action layout

```
my-docker-action/  
  action.yml  
  Dockerfile  
  entrypoint.sh
```

```
# action.yml
name: "My Docker Action"
description: "Runs a containerized task"
inputs:
  greeting:
    required: true
runs:
  using: "docker"
  image: "Dockerfile"
  args:
    - ${ inputs.greeting }
```

Composite action layout

```
# action.yml
name: "My Composite Action"
description: "Runs several existing steps as one action"
runs:
  using: "composite"
  steps:
    - run: echo "step one"
      shell: bash
    - uses: actions/checkout@v4
```

15.3 Workflow Commands Inside an Action

The same workflow-command mechanism from Section 8.2 applies inside an action's own code. A JavaScript action typically uses the official toolkit instead of raw echo commands:

```
const core = require('@actions/core');

try {
  const who = core.getInput('who-to-greet');
  console.log(`Hello, ${who}!`);
  core.setOutput('time', new Date().toTimeString());
} catch (error) {
  core.setFailed(error.message); // non-zero exit -> step fails
}
```

15.4 Versioning Actions

Actions are versioned using Git tags, following Semantic Versioning. Consumers can reference a specific commit SHA (most secure/immutable), a full version tag like @v1.2.3 (most precise), or a rolling major-version tag like @v1 (most common — lets consumers automatically receive non-breaking updates).

- Maintainers publish a real tag (e.g. v1.2.3) for every release.
- They then move a floating major tag (v1) to point at the latest compatible release.
- Consumers pin to @v1 for convenience, or to the exact SHA/tag for maximum reproducibility and supply-chain safety.

15.5 Distribution Models

- **Private** — used only within the repo it lives in, or within an org if shared via internal visibility settings.
- **Public** — any public repository can reference the action with `uses: owner/repo@ref`, without being listed anywhere.
- **GitHub Marketplace** — a public action additionally published and listed for discovery, with a name, icon, color, and category.

15.6 Publishing to the Marketplace

- 1 Action must live in its own public repository with a valid `action.yml` at the root.
- 2 Add a `branding:` block to `action.yml` (icon + color) for the Marketplace listing.
- 3 Create a release/tag for the version you want to publish.
- 4 From the repository's release page, check "Publish this Action to the GitHub Marketplace," choose a category, and accept the Marketplace terms.

NOTE: Best practice for distribution, repeatedly emphasized: always tag semantic releases, keep a floating major-version tag current, write a clear README with inputs/outputs documented, and never bundle secrets or credentials directly into the action's code/image.

16. Reusable Workflows & Enterprise Administration

16.1 Reusable Workflow Templates

Beyond custom actions, GitHub also supports fully **reusable workflows** — an entire workflow file that other workflows can call with `uses:`, sharing complex multi-job pipelines instead of copy-pasting them across repositories. A reusable workflow declares a `workflow_call` trigger and can accept typed inputs/secrets.

```
# .github/workflows/reusable-deploy.yml
on:
  workflow_call:
    inputs:
      environment:
        required: true
        type: string
    secrets:
      DEPLOY_KEY:
        required: true

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - run: echo "Deploying to ${ inputs.environment }"
```

```
# calling workflow
jobs:
  call-deploy:
    uses: my-org/my-repo/.github/workflows/reusable-deploy.yml@main
    with:
      environment: production
    secrets:
      DEPLOY_KEY: ${ secrets.PROD_DEPLOY_KEY }
```

Organizations also use **workflow/starter templates** published to a special .github repository so that any new workflow created across the org starts from an approved, consistent baseline — the org-level analogue of a reusable workflow.

16.2 Enterprise Administration (Domain 4)

At scale, GitHub Actions governance moves from the single-repo settings covered in Section 2.4 up to the organization and enterprise level:

- **Policy enforcement** — an enterprise can lock down which actions/workflows any org beneath it is allowed to run (allow-lists, blocked actions).
- **Centralized self-hosted runner management** — runner groups can be created at the enterprise or org level and shared selectively down to specific repositories.
- **Centralized secrets** — organization-level secrets/variables can be shared across some or all repositories in the org, reducing per-repo duplication.

- **Audit logging** — enterprise/org audit logs record workflow runs, permission changes, and runner registration events for compliance.
- **Spending limits & usage reporting** — admins can cap Actions minutes/storage spend and review usage across every repository in the org.

NOTE: Domain 4 is comparatively light (15%) but is almost entirely about knowing *where* a given control lives: repository settings vs. organization settings vs. enterprise settings, and which of those a given policy can be defined or overridden at.

17. Exam Cheat Sheet & Final Tips

17.1 Key Facts to Memorize

- Exam: 75 questions total, 60 scored + 15 unscored, PSI-delivered (online or test center).
- Domain weights: Author/maintain workflows 40%, Consume workflows 20%, Author/maintain actions 25%, Manage for the enterprise 15%.
- Workflows live in `.github/workflows/*.yml`; jobs run in parallel by default unless linked with needs.
- `GITHUB_TOKEN` is auto-created per run, scoped to the run's repo, and expires with the job.
- Secrets are encrypted and masked in logs; vars are plain-text configuration values.
- Use `$GITHUB_OUTPUT` / `$GITHUB_ENV` / `$GITHUB_PATH` files — not the deprecated `::set-output` / `::set-env` commands.
- `success()` is the implicit default 'if'; use `always()` or `failure()` to run steps after a failure.
- GitHub-hosted runners are ephemeral and billed per-minute; self-hosted runners are persistent and self-managed — avoid self-hosted runners on public repos.
- `actions/cache` speeds up dependency installs across runs; artifacts store run outputs for download/download-in-a-later-job.
- Environments add required reviewers, wait timers, and branch restrictions to gate deployment jobs.
- Three action types: JavaScript (node runtime), Docker (container), Composite (wraps existing steps).
- Version actions with semantic tags; keep a floating major tag (`@v1`) current for consumers.
- Publishing to Marketplace requires a public repo, valid `action.yml`, a release/tag, and a branding block.
- `workflow_dispatch` = manual trigger from the UI; `repository_dispatch` = trigger from an external system via the API.

17.2 Study Strategy Recap

- 1 Watch the full lecture once for the mental model, then re-watch domain-heavy sections (esp. Domain 1 workflow syntax).
- 2 Rebuild every demo hands-on in a scratch GitHub repository — muscle memory beats memorization for YAML structure.
- 3 Drill the object hierarchy (workflow → job → step → action → runner) until it's automatic.
- 4 Take timed practice exams close to test day, and review every missed question against the official domain outline.
- 5 On exam day, don't panic on unusually hard questions — 15 of the 75 are unscored trial questions.

17.3 Where This Document Came From

These notes combine two inputs: on-screen content sampled at roughly one-minute intervals across the full 3-hour-10-minute uploaded video (slide titles, code shown in the editor/terminal, and UI screens from the ExamProCo/Github-Examples repository), and general, independently-verified GitHub Actions knowledge used to fill out full explanations, syntax detail, and worked examples around each topic the video visibly covers. The video's own narration was not transcribed word-for-word; the topic sequence, terminology, and demo flow above follow what the video's slides and on-screen activity actually show.

End of Notes